Tensor Omics (TOX)

Analyze gene expression data and evolutionary relationships between genes

User's Guide

Aaron Schröder, Alexander Schwarzpaul, Anna Beketova, Asis Hallab, Franz-Eric Sill, Jitu Daba, Jonathan Kurz, Laszlo Lang, Luka Faensen, Mohamed Akdi, Stephanie Ped, Vivian Bass Vega

University of Applied Sciences Bingen, Germany

Last revised November 13, 2026

Contents

I. Analysis Recipes 3

1. Detecting Divergent Genes: Outliers and Shift Vectors 4
2. Trajectory-Based Signed Contribution Analysis 8
3. Subfunctionalization, Neofunctionalization and Dosage-Effect Detection (SND) 12

II. Shared Sub-Recipes 16

4. Creating a TOX Dataset 17
5. Normalization of Expression Values 21
6. Gene Family Detection 25
7. Gene Family Centroids 28
8. Clustering Points and Trajectories 31

Part I.

Analysis Recipes

1. Detecting Divergent Genes: Outliers and Shift Vectors

Purpose

Find the genes whose expression has moved away from the rest of their gene family further than the family's own spread explains, and describe *how* each one moved. This is the original Tensor Omics workflow: the distances answer whether a copy diverged, the shift vectors answer in which direction.

Maintainer: Tensor Omics Developers

Required Input Data

- Normalized expression vectors, one gene per column, from the normalization sub-recipe.
- A gene-to-family mapping, with `0` for genes belonging to no family. Genes outside your families may stay in the matrix.
- Family centroids, from the centroid sub-recipe — and the decision made there about which members define them.

Prerequisites / Depends on

- *Normalization of Expression Values* (see chapter 5)
- *Gene Family Centroids* (see chapter 7)

Step-by-Step Workflow

Step 1. Measure each gene against its own family's centroid with `distance_to_centroid`. Genes with no family receive `-1`, a sentinel rather than a distance.

Step 2. Call `detect_outliers`. It does *not* compare distances to a fixed cutoff: it divides each distance by its family's own spread — the Relative Distance Index (RDI) — and flags the top $1 -$ `quantile_level` of those. Scaling per family is what puts a broadly spread family and a tight one on the same footing.

Step 3. Compute the shift vector field with `compute_shift_vector_field`. For each gene it returns two vectors, the family centroid and then the shift $\vec{s} = \vec{p} - \vec{o}$ from that centroid to the gene.

Step 4. Report the flagged genes with their shift vectors. Use the *raw* distances and shifts for everything downstream — the RDI exists to choose which genes are worth looking at, not to describe how far they moved.

Note

The family scaling is not simply each family's own standard deviation. A family with few members cannot give a usable one, so Tensor Omics fits the spread of a family against its mean distance across *all* families with LOESS, and reads each family's scale off that curve. A small family is therefore scaled by the trend its neighbours establish instead of by its own noise.

Example script

Runs the whole chain — centroids, distances, outliers, shift vectors — and writes a per-gene table plus the shift vectors of the flagged genes. Run unchanged from the repository root it takes about two seconds on the bundled data and writes `results/outliers.tsv`.

python/examples/outlier_detection.py

```python
from tensor_omics import (distance_to_centroid, detect_outliers,
                          compute_shift_vector_field)

# --- how far is each gene from its own family's centroid? --
distances = distance_to_centroid(
    expr, centroids, gene_to_family)   # -1 where no family

# --- which of those distances are extreme for their family? -
result = detect_outliers(
    n_families, distances, gene_to_family,
    quantile_level=0.95)               # FRACTION: top 5%
is_outlier = result["is_outlier"]
# an effect size, NOT a p-value:
tail_probability = result["tail_probability"]

# --- in which direction did each gene move? ----------------
field = compute_shift_vector_field(
    expr, centroids, gene_to_family)   # (n_axes, 2, n_genes)
centroid_of = field[:, 0, :]           # the family's centroid
shift       = field[:, 1, :]           # p - o, the divergence
```

Listing 1.1: Fragments to edit: the threshold, and what you keep

Important Parameters

Parameter	Controls	When / which values
`quantile_level`	Where the threshold on the RDI sits; the genes above it are flagged	A *fraction* in [0, 1], not a percentage: `0.95` keeps the top 5%. Lower it to screen more broadly, raise it to report only the strongest divergence
`n_families`	How many families the mapping refers to	Must cover the largest index in `gene_to_fam`, and must be the same number the centroids were computed with
`gene_to_fam`	Which family each gene is measured against	`0` for a gene belonging to no family. Such genes pass through the whole chain without being flagged and without affecting the threshold

Expected Results

With the default `quantile_level=0.95` the flagged fraction is essentially 5% of the genes that have a family — on the bundled data, 512 of 10 234, exactly 5.00%, with 75 594 genes belonging to

no family carried along and none of them flagged. Distances to the own centroid have a median of 1.73 and a maximum of 2.96 there. A flagged fraction far from 1 – `quantile_level` means genes are reaching the test that should not: check for families with a zero centroid first.

⚠ Pitfall: reading the quantile level as a percentage

`quantile_level` is a fraction in $[0, 1]$. Passing `95` does not ask for the top 5% — it is out of range and the call fails. Passing `0.05` silently asks for the top 95%, flagging almost everything.

⚠ Pitfall: treating the tail probability as a p-value

The `tail_probability` output states how extreme a gene's distance is *relative to the distances actually observed* — an empirical upper-tail effect size. It is not a null-hypothesis test: nothing here models what the distances would look like in the absence of divergence, so a `tail_probability` of 0.001 is not a p of 0.001 and must not be corrected for multiple testing as though it were.

⚠ Pitfall: a family whose spread cannot be estimated

A family with a single member *is* its own centroid: the distance is zero, there is no intra-family spread to estimate, its scaling factor is zero, and the gene can never be flagged. That is the right answer — one copy cannot diverge from itself — but it is silent, so a family reduced to one member by filtering leaves the analysis without a word. Check family sizes rather than trusting an empty result. And where too few families remain to fit the trend at all, every family falls back to a single global spread, losing exactly the between-family comparability the RDI exists to provide.

⚠ Pitfall: testing genes against a reference that contains them

If the centroids came from `group_centroid_all`, every candidate is part of the reference it is measured against, which pulls the centroid towards the divergent copies and shrinks exactly the distances this recipe looks for. Prefer orthologs-only centroids when the question is whether a copy diverged — see *Gene Family Centroids* (chapter 7).

⚠ Pitfall: averaging the distance sentinel

`distance_to_centroid` returns `-1` for a gene with no family. That is a marker, not a short distance: summarising the distance column without excluding it drags every statistic down. Select on the family mapping, not on the distances.

🔍 Interpretation

A flagged gene is one that sits further from its family's consensus than that family's own spread makes ordinary. It is a statement about *relative* divergence: the same absolute distance is unremarkable in a widely spread family and striking in a tight one, which is the whole point of scaling per family.

The distance says only *that* a gene moved. The shift vector says *where* to, and that is the interpretable part: its large components name the tissues that carry the divergence. A shift concentrated on one axis is a copy that gained or lost expression in that one tissue; a shift spread evenly across axes is a change in overall level rather than in pattern. Read the shift vectors of the flagged genes before drawing conclusions — and carry the raw shifts, not the RDI, into any later geometric analysis.

Finally, the flagged set is a *ranking cut*, not a discovery set with an error rate. At `quantile_level=0.95` you get the top 5% by construction, whether or not anything in your data has genuinely diverged.

References

- Cleveland (1979). Robust locally weighted regression and smoothing scatterplots. *Journal of the American Statistical Association* 74(368):829–836.

2. Trajectory-Based Signed Contribution Analysis

Purpose

For entities measured repeatedly over time — countries, patients, cell lines — find which measured factors move together with a chosen dependent variable, how strongly over the whole trajectory, and at which individual time points. Contributions are signed, so co-movement and divergence are distinguished rather than conflated.

Maintainer: Tensor Omics Developers

Required Input Data

- Repeated measurements arranged as `(n_factors, n_samples, n_timepoints)` — indicators first, entities second, *time last*. Every entity must be measured at the same time points.
- Which indices are factors and which is the dependent. Both are positions in the first axis: a dependent is simply an indicator you chose to explain.
- Regularly spaced time points, if you intend to use the velocity or acceleration variants. Interpolating irregular sampling is not part of the library.

Prerequisites / Depends on

- None beyond having your measurements in that layout.

Step-by-Step Workflow

Step 1. Optionally min–max each series to $[0, 1]$ with `normalize_all_trajectories`, so entities of very different magnitude become comparable. It rescales each series independently and so preserves shape; check its `status` output, which reports per-series problems rather than raising them.

Step 2. Choose a baseline. Deviations are measured from it, so it decides what "co-movement" means: `baseline_min` for growth-like variables, `baseline_mean` for oscillating ones, `baseline_raw` when zero is itself meaningful.

Step 3. Compute contributions with `compute_all_contributions`: one signed number per (factor, dependent, entity) plus the per-timepoint series behind it. This is descriptive — it ranks factors, it does not test them.

Step 4. Test a pair with `perform_permutation_test` and `compute_p_values`. The test replaces the dependent with another entity's, keeping time order intact, and so asks whether *this* entity's pairing is unusual within your panel.

Step 5. Optionally repeat on the dynamics with `compute_velocity_acceleration_contributions`, which answers whether the two variables change, and change their rate of change, together.

Example script

Builds a synthetic panel in which one entity's dependent really is driven by one of its factors and every other entity is unrelated, then runs the whole chain so the planted answer can be checked against the output. The repository ships no time-series data, which is why the example generates its own.

python/examples/trajectory_contributions.py

```
import numpy as np
from tensor_omics import (compute_all_contributions,
                          compute_contributions,
                          perform_permutation_test,
                          compute_p_values)

# --- indicators first, entities second, TIME LAST ----------
traj = np.asfortranarray(panel)   # (n_factors, n_samples,
                                  #  n_timepoints)

# --- 1-based indices into the first axis -------------------
factors   = np.array([1, 2, 3], dtype=np.int32)
dependent = np.array([4], dtype=np.int32)

res = compute_all_contributions(
    traj, factors, dependent,
    "baseline_mean")        # or _min / _raw
totals = res["total_contributions"]   # (n_f, n_d, n_s)

# --- is entity 1's pairing unusual in THIS panel? ----------
obs = compute_contributions(
    traj[0, 0, :].copy(), traj[3, 0, :].copy(),
    "baseline_mean")
perm = perform_permutation_test(
    traj, 1, 4, 1,          # factor, dependent, sample
    "baseline_mean", 2000, random_seed=5)
p = compute_p_values(
    obs["local_contributions"], obs["total_contribution"],
    perm["local_contributions"], perm["total_contributions"])
```

Listing 2.1: Fragments to edit: the layout, the baseline, the pair you test

Important Parameters

Parameter	Controls	When / which values
`baseline_mode`	Where each variable's deviations are measured from, and therefore what a positive contribution means	`baseline_min` for monotone growth variables, so deviations are improvements above the historical minimum; `baseline_mean` for oscillating ones; `baseline_raw` only when zero is a real reference

Parameter	Controls	When / which values
`n_permutations`	Size of the null distribution, and with it the smallest reportable p	A p can never fall below $1/n$; use at least `1000` if you intend to report values near 0.01
`random_seed`	Makes the permutation draw reproducible	Always set it for anything you will report, otherwise the p changes between runs
`factor_indices` / `dependent_indices`	Which rows of the first axis are treated as explaining and explained	1-based. Nothing distinguishes a "factor" from a "dependent" in the data — the roles are yours

Expected Results

On the example's planted panel, the driving factor separates cleanly from the rest: total contribution 12.7 against 0.85 and −0.56 for the two unrelated factors, with $p < 0.0005$ against about 0.4 and 0.55 (the exact values depend on the compiler's random number generator). If a factor you know to be unrelated scores comparably to one you expect to matter, look first at whether the tested entity's *dependent* differs from the other entities' — see the pitfall below — before concluding anything about the factor.

Pitfall: the test only ever looks at the upper tail

The p-value counts permutations whose contribution is ≥ the observed one. A strongly *negative* contribution — the two variables reliably diverging, which is a finding — therefore lands near $p = 1$, not near 0: a total of −3.76 scores $p = 0.63$. Divergence cannot be declared significant this way. Test the negated factor if you need it.

Pitfall: a p-value of exactly zero

The p is the raw fraction (permuted ≥ observed)$/n$, with no correction, so when no permutation reaches the observed value it is reported as exactly 0. No permutation test can support that. Report it as $< 1/n$ instead, and remember it is a resolution limit, not evidence: with 2000 permutations nothing below 0.0005 is measurable.

Pitfall: what the permutation actually holds fixed

The null swaps in *another entity's dependent*. So the question is never "are these two variables related?" but "is this entity's pairing unusual among the entities I supplied?". Where every entity shares the same temporal trend, a real and strong relationship correctly scores around $p = 0.5$, because a swapped-in dependent carries that trend too. The panel is the null; choose it deliberately.

Pitfall: an entity whose dependent is unlike the rest

The mirror image of the previous trap, and easier to fall into. If the tested entity's dependent behaves differently from the others — a strong trend where the rest are flat — then *every* factor in that entity is being compared against a null built from unlike dependents, and unrelated factors come out significant. In a panel built that way, a pure-noise factor reached $p = 0.022$ and outranked the genuine driver. Check that the dependent is exchangeable across entities before believing any of the p-values.

Pitfall: feeding velocities back into the contribution routines

`compute_velocity_trajectories` returns `(n_timepoints-1, n_factors, n_samples)` — the axes reordered relative to its own input. Passing that to `compute_all_contributions`, which expects `(n_factors, n_samples, n_timepoints)`, raises no error: the extents are simply read in the wrong roles and plausible numbers come back computed over the wrong axes entirely. Use `compute_velocity_acceleration_contributions`, which starts from positions and keeps the layout, unless you are transposing deliberately.

Note

The methods document also describes weighted combinations of factors and of dependents, $F^* = \sum_j \alpha_j F_j$. There is no separate routine for these: build the combined series yourself and pass it as another row of the first axis. Group-level comparisons between sets of entities are likewise left to the user, on the totals this recipe produces.

Interpretation

A total contribution is a *descriptive* quantity: its sign says whether the factor and the dependent deviate from their baselines together or in opposition, and its magnitude how consistently. Ranking factors by it is the primary result, and it stands on its own without any p-value.

The per-timepoint series is where the method earns its name. A total built from one enormous spike and twenty quiet points means something quite different from the same total spread evenly across the trajectory, and only the series distinguishes them — read it before interpreting the total.

Velocity contributions say whether the two variables *change* together, which is a different claim from moving together: two variables can sit at consistently high levels without their year-to-year increments aligning at all. Acceleration contributions mark shared turning points. Both are answers to narrower questions than the ambient contribution, not refinements of it.

Finally, keep the descriptive and the inferential parts apart when reporting. The contribution describes your data; the p-value describes how your entity compares with the other entities you happened to supply.

References

- Good (2005). *Permutation, Parametric, and Bootstrap Tests of Hypotheses*. 3rd ed., Springer.
- Phipson & Smyth (2010). Permutation p-values should never be zero: calculating exact p-values when permutations are randomly drawn. *Statistical Applications in Genetics and Molecular Biology* 9(1):Article 39.

3. Subfunctionalization, Neofunctionalization and Dosage-Effect Detection (SND)

Purpose

After a duplication, decide what became of the copies: whether a subset of them divided the ancestral expression between them (subfunctionalization), whether they kept its direction but amplified it (dosage effect), or whether one of them expresses where the ancestor did not (neofunctionalization). Each is a separate test with its own criterion.

Maintainer: Tensor Omics Developers

Required Input Data

- An ancestral expression vector per family. In practice this is the family's orthologs-only centroid — the conserved reference, not the all-genes mean.
- The paralogs' expression vectors, as columns, and a gene-to-family mapping.
- The angle between each gene and its family's ancestor, in radians. No routine computes these; the filters take them as given.

Prerequisites / Depends on

- *Normalization of Expression Values* (see chapter 5)
- *Gene Family Centroids* (see chapter 7)
- *Gene Family Detection* (see chapter 6)

Step-by-Step Workflow

Step 1. Compute each paralog's angle to its family's ancestor, and pick a threshold on that angle. Subfunctionalization keeps the copies *at or above* it, the dosage effect those *at or below*.

Step 2. Prefilter with `filter_paralogs_by_pattern_subfunctionalization` or `..._dosage_effect`. Each returns one bitmask per family, naming the copies that pattern will consider at all.

Step 3. Size the search with `calc_work_arr_paralog_subsets_size` and use the `max_subset_size` it returns — it caps the value you asked for to what the filtered family can support.

Step 4. Run `detect_subfunctionalization` or `detect_dosage_effect`. Both search subsets of the filtered copies: the first for a subset whose sum lands within `rdi_threshold` of the ancestor, the second for one that stays within `max_angle` of it while exceeding its magnitude by `gain_gamma`.

Step 5. Decode each result with `bit_mask_test`, which reports whether a given 1-based gene index is a member of that subset.

Step 6. Test neofunctionalization separately with `detect_neofunctionalization`, on *RAP-projected unit* vectors. It returns one boolean per gene and axis, so it names the tissues that changed.

Note

The three tests do not share an input space. Subfunctionalization and the dosage effect add expression vectors up, so they need the raw vectors with their magnitudes intact. Neofunctionalization asks only about direction, so it needs vectors projected into the RAP and scaled to unit length, and rejects anything outside $[-1, 1]$. Mixing the two is the most common way to misuse this module.

Example script

Builds one synthetic family containing each pattern exactly once and runs all three detectors over it, so the output can be checked against what was planted. The repository ships no ortholog/paralog designation, so no bundled family could serve.

python/examples/snd_detection.py

```
import numpy as np
from tensor_omics import (
    filter_paralogs_by_pattern_subfunctionalization,
    calc_work_arr_paralog_subsets_size,
    detect_subfunctionalization, bit_mask_test,
    bit_mask_n_words)

# --- angles to the ancestor are YOURS to supply ------------
angles = np.arccos(cosines)        # radians, 0 <= a <= pi
threshold = float(np.median(angles))

n_chunks = bit_mask_n_words(n_genes)
masks = filter_paralogs_by_pattern_subfunctionalization(
    angles, threshold, n_families, gene_to_fam, n_chunks)

# --- one family at a time: its ancestor, its mask ----------
mask = np.asarray(masks)[:, family - 1]
sizing = calc_work_arr_paralog_subsets_size(
    max_subset_size, n_genes, mask)     # it CAPS the size

norms = np.linalg.norm(genes, axis=0)
res = detect_subfunctionalization(
    ancestor, genes, mask, sizing["max_subset_size"],
    rdi_threshold,                      # residual tolerated
    norms, (np.argsort(norms) + 1).astype(np.int32))

subsets = np.asarray(res["work_arr_paralog_subsets"])
for k in range(res["n_results"]):       # decode the bitmasks
    picked = [i for i in range(1, n_genes + 1)
              if bit_mask_test(n_genes, subsets[:, k], i)]
```

Listing 3.1: Fragments to edit: the threshold, and each pattern's criterion

Important Parameters

Parameter	Controls	When / which values
`threshold`	The angle at which the prefilters split wide-angle copies from narrow-angle ones	A single value for *all* families. The methods document describes a per-family median; that is not expressible here, so pick a global quantile of the angle distribution and say which
`rdi_threshold`	How far a subfunctionalization subset's sum may land from the ancestor	The distance below which you consider the ancestral expression reconstructed. Larger admits looser partitions
`max_angle`	How far a dosage subset's sum may point away from the ancestor	**Defaults to π, which is no constraint at all.** The pattern is defined by small angles, so set this deliberately
`gain_gamma`	How much a dosage subset must exceed the ancestor's magnitude	Default `0.1`, i.e. about 10% more. Raise it to demand a clearer amplification
`max_subset_size`	Largest number of copies combined into one candidate	Use the value `calc_work_arr_paralog_subsets_size` returns. The search is combinatorial, so a cap around 15 is advisable for large families

Expected Results

On the example's planted family of five copies, each detector finds exactly what was placed there: subfunctionalization returns the pair whose sum reproduces the ancestor with residual 0.000; the dosage effect returns the pair summing to 1.50× the ancestor's magnitude; and neofunctionalization flags the new-domain copy. It also flags both subfunctionalized copies — see the interpretation, that is correct and not a defect.

Pitfall: giving neofunctionalization raw expression vectors

`detect_neofunctionalization` takes RAP-projected *unit* vectors, for the ancestors, the genes and the per-axis thresholds alike, and rejects values outside $[-1, 1]$ with `ToxInputError: invalid input arguments (argument 'ancestors')`. Project with `omics_vector_RAP_projection` and scale with `normalize_unit_length` first — which works *in place* and returns nothing, so use the array you passed in, not the return value.

Pitfall: an ancestor expressed evenly everywhere

A gene expressed uniformly across all axes lies along the space diagonal, so its RAP projection is the origin and it has no direction to normalize: `normalize_unit_length` then fails with `Division by zero encountered`. Neofunctionalization simply cannot be evaluated against such an ancestor — there is no ancestral preference for a copy to depart from. Detect this case and report it rather than working around it.

Pitfall: leaving the dosage angle at its default

`max_angle` defaults to π, which admits every direction, so the dosage search reduces to "any subset that is large enough". The pattern is defined by copies pointing the *same way* as the ancestor. Set the angle yourself, and record what you set.

Pitfall: one threshold, two inclusive bounds

The prefilters keep angles ≥ the threshold for subfunctionalization and ≤ it for the dosage effect, so a copy sitting exactly at the threshold passes *both* — with the median as threshold and an odd number of copies, the middle one always does. That is harmless, but it means the two candidate sets are not a partition of the family.

Interpretation

Read the three results together; they are not mutually exclusive, and a copy may register in more than one. A copy that took over part of the ancestral expression has by definition changed direction, so neofunctionalization flags it too. What separates the patterns is the evidence each requires: only subfunctionalization asks that the copies *reconstruct* the ancestor between them, only the dosage effect asks that they preserve its direction while amplifying it, and neofunctionalization asks nothing about the other copies at all. A copy flagged by neofunctionalization alone — with no subset containing it reconstructing the ancestor — is the clean case.

A subfunctionalization result is a claim about a *set*, not about single genes: it says these copies together account for the ancestral expression. The residual is how well they do so, and belongs in any report of the result.

Because the search is over subsets, absence of a result is weak evidence. It may mean no such division occurred, or that the prefilter removed a participant, or that `max_subset_size` stopped the search below the size needed. Record all three settings with any negative result.

References

- Force, Lynch, Pickett, Amores, Yan & Postlethwait (1999). Preservation of duplicate genes by complementary, degenerative mutations. *Genetics* 151(4):1531–1545.
- Innan & Kondrashov (2010). The evolution of gene duplications: classifying and distinguishing between models. *Nature Reviews Genetics* 11:97–108.

Part II.

Shared Sub-Recipes

4. Creating a TOX Dataset

> **Purpose**
>
> Read your expression table and gene families into the arrays Tensor Omics works on, check that they agree with each other, and store them together as a `.txdata` archive you can reopen later. Every other recipe starts from the arrays this one produces.

Maintainer: Tensor Omics Developers

Required Input Data

- A gene × sample table (CSV/TSV): one identifier column, then one column per replicate or sample.
- A gene family table in OrthoFinder's `Orthogroups.tsv` layout: one family per row, one column per species, gene ids comma-separated.
- Nothing else — this is the entry point.

> **Prerequisites / Depends on**
>
> - None. Later recipes depend on this one, not the other way round.

> **Note**
>
> A TOX dataset is not an object. It is a handful of plain arrays that agree by *position*: column *i* of `expression` belongs to the gene named by `gene_ids[i]`, and entry *i* of `gene_to_family` names that gene's family in `family_ids`. Nothing enforces this at run time, and nothing owns the arrays — keeping them consistent is yours to do, which is what the validation step below is for.

Step-by-Step Workflow

Step 1. Measure your files first. The readers *fill arrays you allocate*, so the number of genes, the number of samples, the number of families and the width of the longest identifier all have to be known before the first read. Count them from the files.

Step 2. Read the identifiers with `read_gene_ids_from_tsv_file`, then the matrix with `read_expression_vectors_tsv` into an `(n_samples, n_genes)` column-major array you allocated, then the families with `read_orthofinder_file`. A gene in no family gets the sentinel `0`.

Step 3. Check the arrays against each other before you rely on them: identifiers unique, family indices in range, expression non-negative. These are cheap and they catch the mistakes that are silent later.

Step 4. Store what you have with `save_tox_data`. Each array is passed together with the name it gets inside the archive; you may store any subset, so a dataset without centroids yet is perfectly valid.

Step 5. Reopen with `read_tox_data_into`, which returns every member. Members that were never written come back with zero extents.

Example script

Reads the bundled tables with the library's own readers, validates the relationships, writes an archive and reads it back to show the round trip is exact. Run unchanged from the repository root it takes about eight seconds over 88 327 genes and writes `results/example.txdata`.

python/examples/tox_dataset.py

```
import numpy as np
from tensor_omics import (read_gene_ids_from_tsv_file,
                          read_expression_vectors_tsv,
                          read_orthofinder_file, save_tox_data,
                          read_tox_data_into)

# --- sizes FIRST: the readers fill arrays you allocate -----
n_genes, n_samples, id_width  = 88327, 67, 14
n_families, family_id_width   = 15512, 9

gene_ids = read_gene_ids_from_tsv_file(
    "expression.tsv", id_width, n_genes,
    n_header_rows=1, gene_col=1)

# filled in place, so allocate it in its final form
expression = np.zeros((n_samples, n_genes),
                      dtype=np.float64, order="F")
read_expression_vectors_tsv(
    file_list=["expression.tsv"], gene_ids=gene_ids,
    expression_vectors=expression,
    n_header_rows=1, gene_col=1,
    value_cols=np.arange(2, n_samples + 2, dtype=np.int32),
    start_row=1)

families = read_orthofinder_file(
    "Orthogroups.tsv", gene_ids, family_id_width,
    n_families, n_genes)

# --- store: each array WITH the name it gets in the archive
save_tox_data("dataset.txdata",
    gene_ids=gene_ids,   gene_ids_file="gene_ids.bin",
    expression=expression, expression_file="expression.bin")

back = read_tox_data_into("dataset.txdata")
```

Listing 4.1: Fragments to edit: the files, their shape, and what you store

Important Parameters

Parameter	Controls	When / which values
`gene_ids_strlen`	Width of the fixed-size character slot each identifier is stored in	Must be at least the longest identifier; measure it, do not guess. Anything longer is truncated, which silently breaks the match between the expression table and the family table

Parameter	Controls	When / which values
`value_cols`	Which columns of the table hold values, 1-based	`2 ...n_samples+1` for the usual layout of one identifier column followed by the samples. Getting the count wrong leaves rows of the matrix untouched at zero
`n_header_rows`	Rows to skip before the data	1 for a single header line
`*_file`	The name each array is given inside the archive	Required whenever its array is passed, and vice versa. The names are yours to choose; the manifest records which is which

Expected Results

On the bundled data the readers report 88 327 genes × 67 samples and 15 512 families, of which 81 960 genes are assigned to a family and 6 367 carry the unassigned sentinel. All four checks pass, and reading the archive back reproduces every array exactly. `get_tox_data_dims` then reports `gene_id_len=14` and `family_id_len=9` — the widths actually needed. The file is about 49 MB for a 47.3 MB matrix: the archive is a container, not compression, so expect roughly the size of the raw data.

Pitfall: sizing the arrays by guess

Every extent has to be supplied before the read, and a wrong one fails quietly rather than loudly. An identifier width below the longest id truncates identifiers, so genes stop matching between the two tables and the family assignment comes back mostly empty; a `value_cols` shorter than the sample count leaves the remaining rows of the matrix at zero. Derive all of them from the files, as the example script does.

Pitfall: passing an array without its member name

`save_tox_data` takes each array together with the name it gets inside the archive, and needs both. Passing only one half fails with `ToxInputError: invalid input arguments`, naming the missing argument. Do not work around it by dropping the array — that is the data you meant to keep.

Pitfall: assuming the archive validates itself

Nothing checks the arrays against each other on the way in or out. An archive whose `gene_to_family` is a different length from its `gene_ids` stores and reloads without complaint, and the mismatch only surfaces as a wrong answer several recipes later. Validate before storing.

Pitfall: reading a file that is not a TOX archive

The reader reports what went wrong at the file level, not at the schema level: a missing file gives `ToxIOError: could not open file`, an ordinary zip gives `Could not extract file from archive`, and an archive whose manifest lists nothing gives `could not read array data`. None of these means your analysis is wrong — they mean the path is.

Interpretation

What you have at the end is the dataset every other recipe names in its required inputs. The expression matrix here is still *raw*: normalize it before any distance or angle is computed from it, and store the normalized matrix rather than recomputing it, so that every later result refers to one known state of the data.

Treat the archive as the unit of reproducibility. It records the arrays and their agreed positions, but not how they were produced — not the normalization settings, not which centroid convention was used, not the outlier threshold. Keep that alongside it, because a `.txdata` on its own cannot tell you which of the choices in the recipes that follow were made.

References

- Emms & Kelly (2019). OrthoFinder: phylogenetic orthology inference for comparative genomics. *Genome Biology* 20:238.

5. Normalization of Expression Values

Purpose

Turn a replicate-level expression table into the gene × tissue matrix that every Tensor Omics analysis uses as its expression vectors: gene-wise scale removed, replicates averaged per tissue, dynamic range compressed. This is the shared entry point of the TOX pipeline; the geometry of every later recipe is fixed by the choices made here.

Maintainer: Tensor Omics Developers

Required Input Data

- A table of non-negative expression measurements (e.g. TPM from `kallisto`), genes × replicates, one column per biological replicate.
- The replicate layout: how many replicates belong to each tissue, condition or time point. The replicates of one tissue must occupy *adjacent* columns.
- For stress-response designs only: which tissue is the control and which the condition, for each pair you want a fold change of.

Prerequisites / Depends on

- None — this is the first step of every Tensor Omics analysis.

Step-by-Step Workflow

Step 1. Arrange the measurements as an `(n_replicates, n_genes)` column-major array, so that *one gene is one column*. Group the replicates of each tissue into adjacent rows and record the group sizes in `reps_per_tissue`, e.g. `[4, 5, 5]` for three tissues measured with four, five and five replicates.

Step 2. Decide whether to quantile-normalize across replicates (`use_quantile`). It is off by default; switch it on when the replicate distributions differ enough that the difference is technical rather than biological.

Step 3. Run `normalization_pipeline`. It scales each gene by a LOESS-stabilized standard deviation, optionally quantile-normalizes, averages the replicates of each tissue, and finally applies $x \mapsto \log_2(x + 1)$.

Step 4. Check the result against the expectations below before using it: the shape, the value range, and the number of genes that reached the output unscaled.

Step 5. Stress-response designs only: call `calc_fchange` on the *log-transformed* matrix to turn control/condition pairs into log2 fold changes, so each such axis carries e.g. “drought in root over control root” rather than an absolute level.

Example script

Reads a replicate-level TSV, derives the tissue layout from its header, runs the pipeline and writes the normalized matrix. Run unchanged from the repository root, it normalizes the bundled `material/kallisto_sex_data_no_na.tsv` (88 327 genes, 67 replicates, 13 tissues) in a few seconds and writes `results/normalized_expression.tsv`.

python/examples/normalization.py

```
import numpy as np
from tensor_omics import normalization_pipeline

# --- input: one gene per COLUMN, -------------------------
#     replicates of a tissue in adjacent ROWS
expr = np.asfortranarray(raw_counts)  # (n_replicates, n_genes)

# --- tissue layout: one entry per tissue, ----------------
#     summing to n_replicates
reps_per_tissue = np.array([4, 5, 5], dtype=np.int32)

# --- parameters (see the table below) --------------------
normalized = normalization_pipeline(
    expr, reps_per_tissue,
    span=0.7,             # LOESS span, mean-vs-SD fit
    degree=2,             # LOESS polynomial degree
    use_quantile=False,   # quantile normalization
)                         # -> (n_tissues, n_genes)
```

Listing 5.1: Fragments to edit: the input, the tissue layout, the parameters

Note

Tensor Omics stores one expression vector per *column* because Fortran is column-major. A matrix handed over as genes × replicates usually fails with the `Array size mismatch` described below, because its row count is then the gene count. Should the two counts happen to agree, it is not rejected — it is silently interpreted as replicates × genes, and every downstream distance and angle is then meaningless. Check `expr.shape` before you call.

Important Parameters

Parameter	Controls	When / which values
`reps_per_tissue`	How the replicate rows are grouped into tissues; entry *i* is the number of adjacent rows belonging to tissue *i*	Must sum to `n_replicates`, the row count of `expr`, otherwise the call fails with `ToxInputError: Array size mismatch (argument 'expr', via 'n_replicates'); 'n_replicates' must equal sum(reps_per_tissue)`. Replicate counts may differ between tissues

Parameter	Controls	When / which values
`span`	Fraction of the genes entering each local fit of the mean–SD LOESS curve	Keep the default `0.7`. Lower it only if the mean–SD trend is genuinely non-monotonic and the fit visibly oversmooths; too low a span makes the curve follow noise
`degree`	Polynomial degree of the local LOESS fits	Keep the default `2`. Use `1` for a strongly linear trend or very few genes
`use_quantile`	Whether the replicate distributions are forced onto a common distribution after gene-wise scaling	Default `False`. Switch on when replicates differ in distribution for technical reasons; leave off when the between-replicate differences are the signal

Expected Results

The result has shape `(n_tissues, n_genes)` and is read-only — call `.copy()` if you need to modify it. For a non-negative input every value is ≥ 0, since $\log_2(x+1) \geq 0$ there. On the bundled example data the script prints the range $[0, 4.056]$ without quantile normalization and $[1.486 \times 10^{-5}, 4.265]$ with it, and 2 627 of the 88 327 genes (3%) have zero variance across their replicates and therefore reach the output unscaled. A value range wildly different from this, or a much larger unscaled fraction, points at the input layout rather than at biology.

Pitfall: replicates that are not adjacent

`reps_per_tissue` describes *contiguous blocks* of rows, not a grouping by name. If the replicate columns of a tissue are scattered through the table, the pipeline still runs and silently averages the wrong columns together. Sort the columns tissue by tissue first; the example script refuses to continue when it detects a non-adjacent tissue.

Pitfall: genes the standard-deviation fit cannot use

The scaling factor comes from a LOESS curve fitted over all genes, so a gene with zero variance across its replicates carries no mean-versus-SD information. Such genes are left out of the fit *and reach the output unscaled* — they are not dropped and not flagged. Count them, as the example script does. If fewer than five genes have non-zero variance, the whole call fails with `ToxInputError: invalid input arguments`; this is what you get for an all-zero matrix, or for a toy example with too few genes.

Pitfall: expecting quantile normalization by default

Quantile normalization is an *option*, not a pipeline stage: the default is scale → average → $\log_2(x+1)$. Two analyses that differ only in `use_quantile` live in different geometries and must not be compared. Record the flag alongside the results.

Pitfall: fold changes computed on the wrong matrix

`calc_fchange` *subtracts* one axis from another. That is a log2 fold change only if you pass the log-transformed matrix — which is what `normalization_pipeline` returns. Passing raw tissue averages instead yields a plain difference of expression levels that merely looks like a fold change.

Warning

`root_mean_sq_normalization` divides by $\sqrt{\text{mean}(x^2)}$, which is not a standard deviation. It is kept for possible future use and must not be used for normalization; use `normalization_pipeline` or `normalize_by_std_dev`.

Interpretation

After normalization, a gene's coordinates state *where its expression sits across tissues*, not how many transcripts were counted. Gene-wise scaling removes each gene's own expression magnitude, so two genes differing a thousandfold in abundance can be neighbours if their tissue profiles agree; the $\log_2(x+1)$ step then compresses the remaining dynamic range so that no single highly expressed tissue dominates a distance. Distances and angles in this space therefore compare the *shape* of expression across tissues.

Because each choice made above defines a different space, results obtained under different settings answer different questions and generally disagree. Agreement between them is evidence of a robust effect; disagreement is usually informative rather than an error. Report the normalization settings with any result derived from this matrix.

References

- Cleveland (1979). Robust locally weighted regression and smoothing scatterplots. *Journal of the American Statistical Association* 74(368):829–836.
- Cleveland & Devlin (1988). Locally weighted regression: an approach to regression analysis by local fitting. *Journal of the American Statistical Association* 83(403):596–610.
- Bolstad, Irizarry, Åstrand & Speed (2003). A comparison of normalization methods for high density oligonucleotide array data based on variance and bias. *Bioinformatics* 19(2):185–193.

6. Gene Family Detection

Purpose

Group genes across species into families, and decide which members of each family count as orthologs. Tensor Omics consumes both and produces neither: the grouping fixes what every later comparison is made *within*, and the ortholog designation fixes what divergence is measured *against*.

Maintainer: Tensor Omics Developers

Required Input Data

- One protein FASTA per species, reduced to primary transcripts.
- Gene identifiers that match those in your expression table exactly — the two are joined by string equality, nothing else.
- OrthoFinder, installed and run separately. It is not part of Tensor Omics and this recipe does not wrap it.

Prerequisites / Depends on

- None. Everything else in this manual depends on this step.

Step-by-Step Workflow

Step 1. Reduce each proteome to one protein per gene, the primary transcript. Isoforms are the single most common cause of inflated families.

Step 2. Run OrthoFinder on the directory of proteomes, following its own documentation. Nothing about this step is Tensor Omics specific.

Step 3. Take `Orthogroups/Orthogroups.tsv` as the family table: one row per orthogroup, one column per species, the genes of that species listed in the cell. This is what `read_orthofinder_file` parses.

Step 4. Decide which genes are the *orthologs*, which is a separate question — see the pitfall below. OrthoFinder answers it in `Orthologues/`, whose pairwise files mark each relation as one-to-one, one-to-many or many-to-many; the genes appearing alone on both sides of a row are the unambiguous one-to-one orthologs.

Step 5. Check the result against your expression table before using it: family sizes, how many species each family covers, and above all how many of your genes actually landed in a family.

Example script

Surveys an `Orthogroups.tsv`, checks it against an expression table, and derives the one-to-one ortholog list from an `Orthologues/` directory. It covers the handoff only — OrthoFinder itself is not run here.

`python/examples/gene_family_detection.py`

```
# one protein per gene: isoforms inflate every family
# (use your annotation's primary-transcript flag)

orthofinder -f proteomes/ -t 16

# what Tensor Omics reads afterwards:
#   Orthogroups/Orthogroups.tsv
#       the FAMILIES: row = orthogroup, column = species
#   Orthologues/<species>/<a>__v__<b>.tsv
#       the ORTHOLOGY relations: 3 columns, Orthogroup + one
#       per species; a row with ONE gene on each side is a
#       one-to-one ortholog pair
#   Orthogroups/Orthogroups_UnassignedGenes.tsv
#       genes in no family -- they get the sentinel 0 in TOX
```

Listing 6.1: The external step, and what to keep from it

Important Parameters

Parameter	Controls	When / which values
`primary transcripts`	Whether one or many proteins represent each gene	Always reduce first. Every isoform kept becomes an apparent paralog and biases family size, centroids and every divergence test built on them
`species set`	Which genomes define the families	Adding a distant species merges families; removing one can split them. The set is part of the result, not a free parameter — record it
`ortholog rule`	Which family members count as the conserved reference	The one-to-one relations in `Orthologues/` are the strict choice. The single-copy orthogroups file is stricter still, but excludes exactly the duplicated families a divergence analysis is about. State which you used
`inflation (MCL, -I)`	Granularity, if you refine large orthogroups further	Higher (`2.0`–`4.0`) yields smaller, tighter families; lower yields larger, more inclusive ones

Expected Results

The bundled `material/Orthogroups.tsv` holds 15 512 orthogroups over four mammal species, family sizes from 2 to 143 with a median of 4. Of these 12 842 contain all four species and 2 670 are missing at least one, while 4 004 hold more genes than species and therefore contain duplications — those are the families an SND analysis has anything to say about. Against the bundled expression table, 81 960 of 88 327 genes land in a family and 6 367 carry the unassigned sentinel. A far larger unassigned fraction almost always means the identifiers do not match, not that the genes are orphans.

Pitfall: taking an orthogroup for a set of orthologs

They are different things, and Tensor Omics needs both. An *orthogroup* is the set of genes descended

from a single gene in the last common ancestor of *all* the species; *orthologues* are genes descended from a single gene in the ancestor of a *pair* of species; *paralogues* arose by duplication within an orthogroup. `Orthogroups.tsv` therefore gives you families that mix orthologs and paralogs, and reading it does not tell you which is which. `group_centroid_orthologs` asks for exactly that distinction, and it comes from `Orthologues/`, not from the orthogroup table.

⚠ Pitfall: retreating to the single-copy orthogroups

`Orthogroups_SingleCopyOrthologues.txt` lists the families with exactly one gene per species, which makes the ortholog question trivial — every member is one. It is also useless for the analyses in this manual: a family with no duplicated copies has no paralog to test for subfunctionalization, no outlier to detect against its siblings. Use it to sanity-check a pipeline, never as the working family set.

⚠ Pitfall: identifiers that nearly match

The family table and the expression table are joined by exact string equality. A version suffix present in one and stripped in the other, or a transcript identifier against a gene identifier, produces no error anywhere — the genes simply come back unassigned and quietly leave the analysis. The example script reports the assigned count for this reason; check it against what you expect before going on.

⚠ Pitfall: isoforms as paralogs

Several splice isoforms of one gene look exactly like several copies of it. They inflate family sizes, drag centroids towards whichever gene has the most isoforms, and can present as a dosage effect that is purely an annotation artefact. Reduce to primary transcripts before OrthoFinder, not after.

🔍 Interpretation

The family grouping is not a preprocessing detail; it defines the comparison. Every later recipe asks how a gene differs from *its family*, so a family that is too inclusive hides divergence inside itself, and one that is too fragmented has nothing to compare against. When a result looks surprising, the family it came from is the first thing to inspect.

The ortholog designation carries a second, independent decision: what counts as the ancestral state. A centroid over one-to-one orthologs is a claim about conserved expression, and the divergence measured against it is divergence from that conserved state. A centroid over all members measures divergence from the family as it is now, duplications included. Both are legitimate; they answer different questions, and no later recipe can recover which one you meant. Record the species set, the ortholog rule and the OrthoFinder version alongside the results.

References

- Emms & Kelly (2019). OrthoFinder: phylogenetic orthology inference for comparative genomics. *Genome Biology* 20:238.
- Emms & Kelly (2015). OrthoFinder: solving fundamental biases in whole genome comparisons dramatically improves orthogroup inference accuracy. *Genome Biology* 16:157.

7. Gene Family Centroids

Purpose

Reduce each gene family to a single expression vector, its centroid: the reference that later recipes measure divergence against. Distances to the centroid, shift vectors from it, and the outlier test built on those all inherit whichever set of family members you average here.

Maintainer: Tensor Omics Developers

Required Input Data

- Normalized expression vectors, one gene per column (`(n_axes, n_genes)`), from the normalization sub-recipe.
- A gene-to-family mapping: for each gene, the index of its family, or `0` for a gene that belongs to none.
- Optional, and only for the orthologs-only centroid: which genes are the orthologs. This is *your* data — it comes from an orthology pipeline (OrthoFinder, synteny/Cactus, OMAmer), not from Tensor Omics.

Prerequisites / Depends on

- *Normalization of Expression Values* (see chapter 5)
- *Gene Family Detection* (see chapter 6)

Step-by-Step Workflow

Step 1. Number the families $1 \ldots n$ and build the `gene_to_family` vector, one entry per column of the expression matrix. Use `0` for every gene that belongs to no family — it is the sentinel the library expects, and such genes are skipped.

Step 2. Choose which members define the reference. `group_centroid_all` averages every gene of the family; `group_centroid_orthologs` averages only the genes marked in `ortholog_set`. See the interpretation below — this choice is the substance of the step, not a detail.

Step 3. Compute the centroids. The result is an `(n_axes, n_families)` matrix holding *one centroid per column*, in the same layout and orientation as the expression vectors that went in.

Step 4. Count, per family, how many genes actually contributed. A family with no contributor is not reported by the library — it silently receives a zero centroid, which is a valid point in the space and will quietly distort everything measured against it.

Example script

Builds the gene-to-family mapping from an OrthoFinder-style family table, computes the centroids and writes them, reporting assignment coverage and any family left without a contributing gene. Run unchanged from the repository root, it computes all-genes centroids for the 458 bundled families over `material/normalization.tsv` and writes `results/family_centroids.tsv`; pass `--orthologs` with a list of gene ids to take the orthologs-only centroid instead.

`python/examples/gene_family_centroids.py`

```
import numpy as np
from tensor_omics import (group_centroid_all,
                          group_centroid_orthologs,
                          max_omp_threads)

# --- one gene per COLUMN, as normalization produced it -----
expr = np.asfortranarray(normalized)   # (n_axes, n_genes)

# --- family index per gene, 1-based; 0 = no family ---------
gene_to_family = np.array([1, 1, 2, 0, 2], dtype=np.int32)
n_families = 2

# --- (a) the family's actual mean, paralogs included -------
centroids = group_centroid_all(
    expr, gene_to_family, n_families, max_omp_threads())

# --- (b) a conserved reference: orthologs only -------------
ortholog_set = np.array([True, False, True, False, True])
centroids = group_centroid_orthologs(
    expr, gene_to_family, n_families, max_omp_threads(),
    ortholog_set)
#   -> (n_axes, n_families), one centroid per column
```

Listing 7.1: Fragments to edit: the mapping, and which members define the centroid

> **Note**
>
> For a centroid over an arbitrary set of genes — not a whole family — call `mean_vector(expr, gene_indices)` directly. Its `gene_indices` are 1-based column numbers, not a boolean mask.

Important Parameters

Parameter	Controls	When / which values
`gene_to_family`	Which family each gene belongs to; entry *i* is the family index of column *i* of the expression matrix	Values must be `1`…`n_families`, or `0` for an unassigned gene. Any other value fails the call
`n_families`	How many centroids are produced, i.e. the number of columns of the result	Must be at least as large as the largest index in `gene_to_family`. Indices above it fail the call; families with no genes below it come back as zero centroids
`n_omp_threads`	The most threads the call may share the families between	At least 1; `max_omp_threads()` is the build's default. A build without OpenMP runs on one thread whatever is passed
`ortholog_set`	Which genes contribute when the orthologs-only centroid is taken	Required by `group_centroid_orthologs`, one boolean per gene. A family with no ortholog among its members gets a zero centroid

Expected Results

The result is `(n_axes, n_families)`, read-only, and each column is the element-wise mean of the contributing genes — so every centroid lies inside the range its family's members span on each axis. On the bundled data the script assigns 10 234 of 85 828 genes to 458 families (the rest belong to none), family sizes run from 6 to 143 genes, and no family is left empty. It also reports the 361 family members that have no row in the expression table; a much larger number there means the gene identifiers of the two files do not match.

Pitfall: a family with no members gets a zero centroid

A family that no gene contributes to — because it has no members in the expression table, or none of them is an ortholog — does not raise an error and is not flagged. It receives the zero vector, which is an ordinary point at the origin of the space. Every later distance to that centroid is then simply the gene's own norm, so the family looks maximally divergent for a purely bookkeeping reason. Count the contributors per family and check the count is never zero before going on.

Pitfall: which members you average changes every later result

The two entry points are not variants of one number. On the bundled data the orthologs-only centroids shipped in `material/centroids_orthologs_only.tsv` sit a median 13.5% of their own length away from the all-genes centroids of the same families, and up to 1.56 units away — both computed from the same expression table, so the whole difference comes from the member set. Averaging *all* genes folds the very paralogs you may later test for divergence into the reference they are tested against, which pulls the reference toward them and hides them. Use the orthologs-only centroid whenever the analysis asks whether a copy has diverged.

Pitfall: marking unassigned genes with anything but zero

`0` is the sentinel for "no family", and genes carrying it are skipped. A negative index, or an index above `n_families`, is not treated as "unassigned" but fails the call with `ToxInputError: invalid input arguments (argument 'gene_to_family')`. Map every gene you want excluded to `0` explicitly rather than leaving a placeholder behind.

Interpretation

A centroid is the family's consensus expression profile, and the distance of a member from it is that member's divergence from the family. What the consensus *means* is decided by the member set. The orthologs-only centroid is a *conserved* reference: it describes where the family's expression sat before the copies you are testing diverged, which is what makes a large distance interpretable as divergence of that copy. The all-genes centroid is the family's actual centre of mass, including every divergent copy — the right reference when you want to describe the family as it is, and the wrong one when you want to detect a member that moved.

Read a centroid alongside its contributor count. A centroid over two genes is a midpoint, not a consensus, and distances measured against it carry the noise of those two genes.

References

- Emms & Kelly (2019). OrthoFinder: phylogenetic orthology inference for comparative genomics. *Genome Biology* 20:238.

8. Clustering Points and Trajectories

Purpose

Group things that behave alike: genes by expression vector, samples by whole time course, or anything else you can place in the space or give pairwise distances for. Two ordinary methods — k-means and agglomerative linkage — with the conventions Tensor Omics wraps them in.

Maintainer: Tensor Omics Developers

Required Input Data

- For k-means: points as `(n_dims, n_points)`, one per column, and an initial centroid per cluster. There is no seeding routine — supplying the centroids *is* the seeding decision, and their count is k.
- For clustering whole trajectories: `(n_factors, n_samples, n_timepoints)`, and one centroid per cluster of length `n_factors×n_timepoints`.
- For linkage: a symmetric distance matrix with a zero diagonal and no negative entries. Computing it is not part of this step.

Prerequisites / Depends on

- None in the library. If you are clustering expression, normalize first — otherwise the grouping follows overall magnitude.

Step-by-Step Workflow

Step 1. Decide what a *point* is. Clustering genes, one column is a gene. Clustering samples by their evolution, one whole trajectory is one point — that is `cluster_factor_trajectories_k_means`, and it is a different question from grouping the individual (sample, timepoint) states, which is plain `k_means_clustering` over the same values.

Step 2. For k-means, choose k and the initial centroids together, since the array you pass carries both. Run `k_means_clustering`; the centroid array comes back holding the final centroids.

Step 3. For linkage, compute the distance matrix yourself and pass it with a criterion. The same matrix can be re-clustered under another criterion without recomputing it, and comes back unchanged either way.

Step 4. Read the merge sequence. Each step reports the two things merged, the height at which they met, and the size of the resulting cluster. Cut the tree at a height of your choosing — the library does not cut it for you.

Example script

Clusters three planted blobs with k-means, groups four trajectories of which two rise and two fall, and prints an average-linkage tree merge by merge with the sign convention decoded. All data is synthetic and separable, so the expected grouping is obvious.

`python/examples/clustering.py`

```python
import numpy as np
from tensor_omics import k_means_clustering, linkage_clustering

# --- k-means: the centroid array carries BOTH k and the seeds
points = np.asfortranarray(data)      # (n_dims, n_points)
centroids = np.asfortranarray(seeds)  # (n_dims, k)
res = k_means_clustering(points, centroids, max_iterations=300)
labels = res["labels"]                # 1-based, one per point
# centroids now holds the FINAL centroids: modified in place

# --- linkage: your distance matrix, their criterion --------
tree = linkage_clustering(
    np.asfortranarray(distances),     # symmetric, zero diagonal
    "average")                        # or "weighted" / "ward"

# positive entry = data point, NEGATIVE = the node made at
# that step. This is the opposite of R's hclust.
for k in range(len(tree["heights"])):
    i, j = tree["merge_i"][k], tree["merge_j"][k]
    height = tree["heights"][k]
```

Listing 8.1: Fragments to edit: k and its seeds, and the linkage criterion

Important Parameters

Parameter	Controls	When / which values
`centroids`	Both the number of clusters and where the search starts	k is the number of columns. The values may be real data points or arbitrary — but they must be distinct, which is *not* checked
`max_iterations`	When k-means gives up short of convergence	Default `300`. Raise it only if you have reason to think it is stopping early; a k-means that needs many more usually has a seeding problem instead
`method`	The linkage criterion, i.e. how the distance between two clusters is defined	`average`, `weighted` or `ward`. Note what is *not* here: `single` and `complete` are rejected

Expected Results

On the example's three separated blobs, k-means recovers them exactly (12/12/12) and the final centroids sit within 0.1 of the planted ones. The four trajectories split two and two, rising against falling. The linkage heights agree with a standard implementation — they match `scipy.cluster.hierarchy.linkage` to floating-point tolerance for all three criteria — and the last merge of the same eight points comes at 0.863 under `average`, 1.069 under `weighted` and 1.134 under `ward`, which is a useful reminder that heights are comparable within a criterion and not across them.

Pitfall: the seeding is yours, and duplicates are not caught

There is no k-means++ here, and no check that your initial centroids differ. Passing the same point k

times is accepted and returns k populated clusters — an arbitrary split of the data with no meaning at all. Seed from distinct data points, and since the outcome depends on that seeding, run it more than once from different seeds before believing a partition.

⚠ Pitfall: the centroid array is overwritten

`centroids` is modified in place: the array you pass in comes back holding the *final* centroids, which is how you retrieve them. Keep a copy first if you wanted your seeds afterwards, and do not reuse the same array across runs expecting the same starting point.

⚠ Pitfall: the merge sign convention is the reverse of R's

In the merge arrays a *positive* entry is a data point and a *negative* entry is the inner node created at that step, so −2 means whatever step 2 produced. R's `hclust` uses exactly the opposite convention, negatives for singletons. Code ported from R will build a mirrored tree without any error being raised.

⚠ Pitfall: expecting the distance matrix to be consumed

The matrix is declared as modified in place, and its lower triangle really is used as scratch — but it is restored from the upper triangle before the call returns, on success and on error alike. There is no need to copy it defensively. It must, however, genuinely be a distance matrix: asymmetry, a negative entry or a non-zero diagonal each fail the call with `ToxInputError: invalid input arguments (argument 'distances')`.

🔍 Interpretation

The two methods answer differently shaped questions. k-means returns a partition at the k you chose and says nothing about whether k was right: every point gets a label, and a run on structureless data returns k tidy clusters just the same. Linkage returns a whole hierarchy and leaves the cut to you, so the height at which groups merge is itself the evidence — a merge far above the ones below it marks a real separation, and a tree that merges at steadily rising heights is telling you there is no natural number of groups.

Whichever you use, the clustering describes the space you built, not biology directly. Cluster raw expression and you group by overall magnitude; cluster normalized vectors and you group by expression shape; cluster trajectories and you group by how things developed rather than where they ended. The choice made in the earlier recipes decides what your clusters mean.

References

- Lloyd (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory* 28(2):129–137.
- Ward (1963). Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association* 58(301):236–244.
- Müllner (2011). Modern hierarchical, agglomerative clustering algorithms. *arXiv*:1109.2378.